

Criterion C

3.1: Project Structure	2
3.2: Qt library	3
3.3: SQLite	4
3.4: OpenAI API	5
3.5: Spaced Repetition Scheduling Algorithm	6
3.6: SQL Integration	7
1. Review_Schedule Table	
2. Adding word to a list	
3. Updating review schedule	
3.7: GUI Elements	10
1. Allowing the user to study random words	
2. Applying new rating to a word	
3. Multiple choice study session	
3.8: Bibliography	13

3.1: Project Structure

There was a lack of folder structure within the actual project directory itself to allow for simpler library linkages and to not overcomplicate the build directory. The file structure was also handled by the QMake build system, with the default .pro file organizing the files within the Qt Creator IDE. Each major part of the application, such as the database wrapper class, main window, and any subsequent windows each had a class which means a header and an implementation file. The only difference is the usage of the database wrapper class for easier updates of the database without having to change individual classes. For the AI element of this application I decided to use OpenAIs API because of its easy implementation through the Qt network library

3.2: Qt Library¹

Qt is a framework for C++ that allows for the creation of GUI elements through their own vast system of libraries. It uses object oriented programming, representing GUI elements through objects in classes of windows. In order to use the Qt framework, I had to install the full Qt library and the Qt Creator to ensure that library linkage and project building would work appropriately.

Qt has many classes and functions on its own, including its system of slots that allow for communication between different windows using signals. Apart from that, the use of pointers allows for careful management of different GUI parts, and when they are displayed. This library allows for the implementation of a simple GUI for the user, satisfying success criteria 9.

```
MainWindow::MainWindow(QWidget *parent)
: QMainWindow(parent)
, ui(new Ui::MainWindow)
, db(getApplicationDataPath("vocabulary.db").toStdString())
{
    ui->setupUi(this);

    // Create panel widgets
    deckListPanel = new DeckListPanel(&db, this);
    modeSelectorPanel = new ModeSelectorPanel(this);
    studyPanel = new StudyPanel(&db, this);

    // Add panels to the main layout
    ui->deckListContainer->layout()->addWidget(deckListPanel);
    ui->modeSelectorContainer->layout()->addWidget(modeSelectorPanel);
    ui->studyPanelContainer->layout()->addWidget(studyPanel);

    // Connect panel signals
    connect(deckListPanel, &DeckListPanel::deckDoubleClicked, this, &MainWindow::onDeckDoubleClicked);
    connect(modeSelectorPanel, &ModeSelectorPanel::startStudyClicked, this, &MainWindow::onStartStudy);
    connect(modeSelectorPanel, &ModeSelectorPanel::viewAllClicked, this, &MainWindow::onViewAll);
    connect(modeSelectorPanel, &ModeSelectorPanel::deleteListClicked, this, &MainWindow::onDeleteList);
    connect(studyPanel, &StudyPanel::studyCompleted, this, &MainWindow::onStudyCompleted);

    // Initial visibility - show only deck list on startup
    showDeckList();

    // Apply initial theme (light mode by default)
    applyLightTheme();
}
```

¹ Qt Framework, Qt Company, last modified 2023, accessed February 21, 2026, <https://www.qt.io/>.

3.3: SQLite²

SQLite is a local database that comes with a simple library which I had to download and link within my project. Qt already has an existing SQL module within its framework, however it was inadequate for what I wanted to accomplish. Using SQLite directly allowed me to have more control over the database directly, making debugging easier with less abstractions in the way. Apart from that, because this application utilizes a single database type, it was better to implement it directly with SQLite. The direct implementation also allows for more specific error handling because of standard conventions when using the SQLite library, which require a pointer to an error message so that it can be thrown in case of an exception.

```
std::vector<DataBase::DueCard> DataBase::getDueCards(int listID) {
    std::vector<DueCard> out;
    const char* sqlAll =
        "SELECT rs.schedule_id, rs.word_id, rs.list_id, w.word, w.definition, rs.ease_factor, rs.interval_days, rs.repetition_count, 1
        FROM review_schedule rs JOIN words w ON rs.word_id = w.word_id "
        "WHERE rs.next_review_date <= datetime('now') "
        "ORDER BY rs.next_review_date ASC;";

    const char* sqlList =
        "SELECT rs.schedule_id, rs.word_id, rs.list_id, w.word, w.definition, rs.ease_factor, rs.interval_days, rs.repetition_count, 1
        FROM review_schedule rs JOIN words w ON rs.word_id = w.word_id "
        "WHERE rs.list_id = ? AND rs.next_review_date <= datetime('now') "
        "ORDER BY rs.next_review_date ASC;";

    const char* sql = (listID < 0) ? sqlAll : sqlList;
    sqlite3_stmt* stmt = nullptr;
    int rc = sqlite3_prepare_v2(db, sql, -1, &stmt, nullptr);
    if (rc != SQLITE_OK) {
        QString errorMsg = "Failed to prepare statement for getDueCards: " + QString::fromStdString(std::string(sqlite3_errmsg(db)));
        qCritical() << errorMsg;
        throw std::runtime_error(errorMsg.toStdString());
    }

    if (listID >= 0) sqlite3_bind_int(stmt, 1, listID);

    while ((rc = sqlite3_step(stmt)) == SQLITE_ROW) {
        DueCard c;
        c.schedule_id = sqlite3_column_int(stmt, 0);
        c.word_id = sqlite3_column_int(stmt, 1);
        c.list_id = sqlite3_column_int(stmt, 2);
        const unsigned char* wtxt = sqlite3_column_text(stmt, 3);
        c.word = wtxt ? reinterpret_cast<const char*>(wtxt) : std::string("");
        const unsigned char* dtxt = sqlite3_column_text(stmt, 4);
        c.definition = dtxt ? reinterpret_cast<const char*>(dtxt) : std::string("");
        c.ease_factor = sqlite3_column_double(stmt, 5);
        c.interval_days = sqlite3_column_int(stmt, 6);
        c.repetition_count = sqlite3_column_int(stmt, 7);
        const unsigned char* ndtxt = sqlite3_column_text(stmt, 8);
        c.next_review_date = ndtxt ? reinterpret_cast<const char*>(ndtxt) : std::string("");
        out.push_back(std::move(c));
    }

    sqlite3_finalize(stmt);
    return out;
}
```

² SQLite Main Page," SQLite, last modified January 9, 2026, accessed February 21, 2026, <https://sqlite.org/>.

3.4: OpenAI API³

OpenAI was the best option not only because their API is well documented and can be easily implemented, but because their models allow for fine tuning control, which is perfect when a task requires something specific such as the generation of a vocabulary list. OpenAI is also reliable and works flawlessly most times, especially with very little downtime. Other AI APIs were possible, such as Google's Gemini and Anthropic's Claude. However, Gemini isn't as easy to integrate on its own without Google and Claude has a stronger focus on other fields that do not benefit vocabulary generation.

```
QJsonObject body;
body["model"] = model;
body["messages"] = messages;
body["max_tokens"] = 1500;
body["temperature"] = 0.8;

QNetworkRequest request(QUrl("https://api.openai.com/v1/chat/completions"));
request.setRawHeader("Authorization", QString("Bearer %1").arg(apiKey).toUtf8());
request.setHeader(QNetworkRequest::ContentTypeHeader, "application/json");

QJsonDocument doc(body);
QByteArray payload = doc.toJson();
ui->generateButton->setEnabled(false);
ui->statusLabel->setText("Generating...");

manager->post(request, payload);
}
```

³ "OpenAI API," OpenAI, last modified June 2020, accessed February 21, 2026, <https://openai.com/index/openai-api/>.

3.5: Spaced Repetition Scheduling Algorithm⁴

```

void SpacedRepetitionCalculator::calculateNextReview(int quality) {
    quality = std::max(0, std::min(5, quality));

    easinessFactor = easinessFactor + (0.1 - (5 - quality) * (0.08 + (5 - quality) * 0.02));

    if (easinessFactor < SpacedRepetitionCalculator::MIN_EF) {
        easinessFactor = SpacedRepetitionCalculator::MIN_EF;
    }
    if (easinessFactor > SpacedRepetitionCalculator::MAX_EF) {
        easinessFactor = SpacedRepetitionCalculator::MAX_EF;
    }

    if (quality < 3) {
        repetitions = 0;
        interval = 1;
    } else {
        repetitions++;

        if (repetitions == 1) {
            interval = 1;
        } else if (repetitions == 2) {
            interval = 6;
        } else {
            interval = static_cast<int>(interval * easinessFactor);
        }
    }

    nextReview = time(nullptr) + (interval * 24 * 3600);
}

```

This is the function within the SpacedRepetitionCalculator class that calculates the next date for review of a word within a list. It takes the quality of a word, or how easy the user found it to review the word, and calculates 3 key variables: the easiness factor, the interval, and the next review date. The easiness factor requires the quality value to be between 0 and 5. The beginning of the function validates the input by making sure that if the input is larger than 5 it gets lowered to 5 and if it is lower than 0 it gets increased to 0. The function also validates the easinessFactor by making sure it does not exceed accepted values, which are stored as member variables within the SpacedRepetitionCalculator class.

⁴ "SuperMemo 1.0 for DOS (1987)," SuperMemoGuru, accessed February 20, 2026, [https://supermemo.guru/wiki/SuperMemo_1.0_for_DOS_\(1987\)](https://supermemo.guru/wiki/SuperMemo_1.0_for_DOS_(1987)).

3.6: SQL Integration

1. Review_schedule table

```
const char* sql =
    "CREATE TABLE IF NOT EXISTS review_schedule ( "
    "schedule_id INTEGER PRIMARY KEY AUTOINCREMENT, "
    "word_id INTEGER NOT NULL UNIQUE, "
    "list_id INTEGER, "
    "next_review_date DATETIME NOT NULL, "
    "ease_factor REAL CHECK (ease_factor BETWEEN 1.3 AND 2.5), "
    "interval_days INTEGER NOT NULL DEFAULT 0, "
    "repetition_count INTEGER NOT NULL DEFAULT 0, "
    "FOREIGN KEY (word_id) REFERENCES words(word_id), "
    "FOREIGN KEY (list_id) REFERENCES vocabulary_lists(list_id) "
    ");";
```

This is the SQL code that creates the review_schedule table, which is responsible for keeping track of all of the data associated with a word and the calculation of the next review date. There are several validation checks, such as ensuring that word_id and next_review_date are NOT NULL. This forces a schedule_id to always have a corresponding word and a review date to make sure the user will at some point study it. The ease_factor is checked to be between 1.3 and 2.5 because that is the range specific to the super memo algorithm.⁵ The table also has foreign keys referenced from the words table and the vocabulary_lists table also ensures data validity by forcing a record to exist.

⁵ "SuperMemo 1.0 for DOS (1987)," SuperMemoGuru.

2. Adding word to a list

```
try {
    beginTransaction();

    int wordID = addOrGetWord(word, partOfSpeech, definition, language);
    if (wordID < 0) {
        rollbackTransaction();
        QString errorMsg = "Failed to obtain or create word id in addWordAndSetup";
        qCritical() << errorMsg;
        throw std::runtime_error(errorMsg.toStdString());
    }

    bool inserted = addWordToList(listID, wordID);

    // Initialize review schedule regardless (INSERT OR IGNORE inside)
    initReviewSchedule(wordID, listID);

    commitTransaction();
    return wordID;
} catch (...) {
    try { rollbackTransaction(); } catch (...) {}
    throw; // rethrow original exception
}
```

This try-catch block ensures thorough error handling when adding a word to a list. Within the try block, it checks whether the word id is valid and throws an error if it's not. Otherwise the program will catch any exceptions thrown after the try block and undo anything added to the list because it wasn't valid.

3. Updating review schedule

```

sqlite3_bind_int(stmt, 1, repetition_count);
sqlite3_bind_int(stmt, 2, interval_days);
// Ensure we don't violate the CHECK constraint on ease_factor (1.3 - 2.5)
double ef = ease_factor;
if (ef < 1.3) ef = 1.3;
if (ef > 2.5) ef = 2.5;
sqlite3_bind_double(stmt, 3, ef);
sqlite3_bind_text(stmt, 4, next_review_date.c_str(), -1, SQLITE_TRANSIENT);
sqlite3_bind_int(stmt, 5, wordID);
sqlite3_bind_int(stmt, 6, listID);

rc = sqlite3_step(stmt);
if (rc != SQLITE_DONE) {
    sqlite3_finalize(stmt);
    QString errorMsg = "Execution failed for updateReviewScheduleForWord: " + QString::fromStdString(std::string(sqlite3_errmsg(db)));
    qCritical() << errorMsg;
    throw std::runtime_error(errorMsg.toStdString());
}

sqlite3_finalize(stmt);
return true;

```

This section of the `updateReviewSchedule` function validates data that is passed to the function, making sure that the easiness factor is within the correct range before binding it for updating the database. There is then further error handling in case there is an error with the database, and the resulting message isn't `SQLITE_DONE`.

3.7: GUI Elements

1. Allowing the user to study random words

```

11 (cards.empty()) {
    QMessageBox::StandardButton reply;
    reply = QMessageBox::question(this, "No Due Cards",
                                "No cards are due for review right now.\n\n"
                                "Would you like to practice random words from this deck?",
                                QMessageBox::Yes | QMessageBox::No);

    if (reply == QMessageBox::Yes) {
        // Load random practice cards
        auto allWords = db.getWordsInList(listID);

        if (allWords.empty()) {
            QMessageBox::information(this, "No Words", "This deck has no words to practice.");
            return;
        }

        // Create DueCard objects from all words (shuffle them)
        std::vector<DataBase::DueCard> allCards;
        for (const auto &wordTuple : allWords) {
            DataBase::DueCard card;
            card.word_id = std::get<0>(wordTuple);
            card.word = std::get<1>(wordTuple);
            card.definition = std::get<2>(wordTuple);
            card.list_id = listID;
            card.ease_factor = 2.5;
            card.interval_days = 0;
            card.repetition_count = 0;
            card.next_review_date = "";
            card.schedule_id = -1;
            allCards.push_back(card);
        }

        // Shuffle the cards
        std::random_device rd;
        std::mt19937 g(rd());
        std::shuffle(allCards.begin(), allCards.end(), g);

        // Take up to 20 cards for practice session
        size_t practiceSize = std::min(size_t(20), allCards.size());
        cards.clear();
        for (size_t i = 0; i < practiceSize; ++i) {
            cards.push_back(allCards[i]);
        }
    }
}

```

This is the logic behind the option for the user to study words in a random order if no words are due in a list. It utilizes the mt19937 random number generator which is standard in C++ for random generation. This allows for easy changes to be made to the number of cards the user can study at once,

2. Applying new rating to a word

```
// In random practice mode, don't update review schedule
if (!isRandomPractice) {
    // prepare calculator with current schedule
    SpacedRepetitionCalculator calc;
    calc.setEasinessFactor(c.ease_factor);
    calc.setRepetitions(c.repetition_count);
    calc.setInterval(c.interval_days);

    calc.calculateNextReview(quality);

    // build next_review datetime string in format YYYY-MM-DD HH:MM:SS
    time_t nextT = calc.getNextReview();
    struct tm tm;
    gmtime_r(&nextT, &tm);
    char buf[64];
    strftime(buf, sizeof(buf), "%Y-%m-%d %H:%M:%S", &tm);
    std::string nextReviewStr(buf);

    // update DB
    bool was_correct = quality >= 3;
    try {
        db->updateReviewScheduleForWord(c.word_id, c.list_id, calc.getRepetitions(), calc.getInterval(),
        std::string modeStr = (studyMode == StudyMode::Flashcard) ? "flashcard" : "multiple_choice";
        db->recordStudySession(c.word_id, c.list_id, was_correct, quality, modeStr);
    } catch (const std::exception &ex) {
        QMessageBox::critical(this, "DB Error", QString::fromStdString(ex.what()));
    }
} else {
    // In random practice mode, just record the session without updating schedule
    bool was_correct = quality >= 3;
    try {
        std::string modeStr = (studyMode == StudyMode::Flashcard) ? "flashcard" : "multiple_choice";
        db->recordStudySession(c.word_id, c.list_id, was_correct, quality, modeStr);
    } catch (const std::exception &ex) {
        QMessageBox::critical(this, "DB Error", QString::fromStdString(ex.what()));
    }
}
```

This function updates the data for a word after it is studied, only after ensuring that it was not a random practice. But because random studying is still studying for the user and it should show up in the statistics, it will still be recorded as a study session just without updating any word schedules. Because the time calculated using spaced repetition algorithm comes as through Unix, the function utilizes gmtime and strftime to reformat the time in a YYYY-MM-DD string for storage within the database.

3. Multiple choice study session

```
// prepare choices: correct + 3 distractors from DB
std::vector<std::string> options;
std::string correctText = c.definition.empty() ? c.word : c.definition;
options.push_back(correctText);

try {
    auto distractors = db->getRandomWordsInList(currentStudyListID, c.word_id, 3);
    for (auto &p : distractors) {
        std::string d = p.second.empty() ? std::string("") : p.second;
        if (d.empty()) d = std::to_string(p.first);
        options.push_back(d);
    }
} catch (...) {
    // ignore DB errors, continue with whatever options we have
}

// if not enough distractors, pad with empty strings
while (options.size() < 4) options.push_back(std::string(""));

// shuffle options and assign to buttons
std::random_device rd;
std::mt19937 g(rd());
std::shuffle(options.begin(), options.end(), g);

correctChoiceIndex = -1;
for (int i = 0; i < 4; ++i) {
    choiceButtons[i]->setText(QString::fromStdString(options[i]));
    choiceButtons[i]->setVisible(true);
    choiceButtons[i]->setEnabled(true);
    choiceButtons[i]->setStyleSheet("");
    if (options[i] == correctText) correctChoiceIndex = i;
}
```

This is part of the display card function which displays a word to the user based on the chosen study method. This is the specific part for multiple choice study, which requires the aggregation of 3 other answers apart from the correct answer. These answers are retrieved and added to a vector, which is then shuffled using the same mt19937 random number generator before being displayed to the user to allow for unpredictability for the user.

Bibliography

OpenAI. "OpenAI API." Last modified June 2020. Accessed February 21, 2026.
<https://openai.com/index/openai-api/>.

Qt Company. Qt Framework. Last modified 2023. Accessed February 21, 2026.
<https://www.qt.io/>.

"SQLite Main Page." SQLite. Last modified January 9, 2026. Accessed February 21, 2026.
<https://sqlite.org/>.

"SuperMemo 1.0 for DOS (1987)." SuperMemoGuru. Accessed February 20, 2026.
[https://supermemo.guru/wiki/SuperMemo_1.0_for_DOS_\(1987\)](https://supermemo.guru/wiki/SuperMemo_1.0_for_DOS_(1987)).